

PROGRAMACIÓN DE MICROCONTROLADORES

ADC

**Carrera de Especialización en Sistemas Embebidos
Facultad de Ingeniería - UBA**

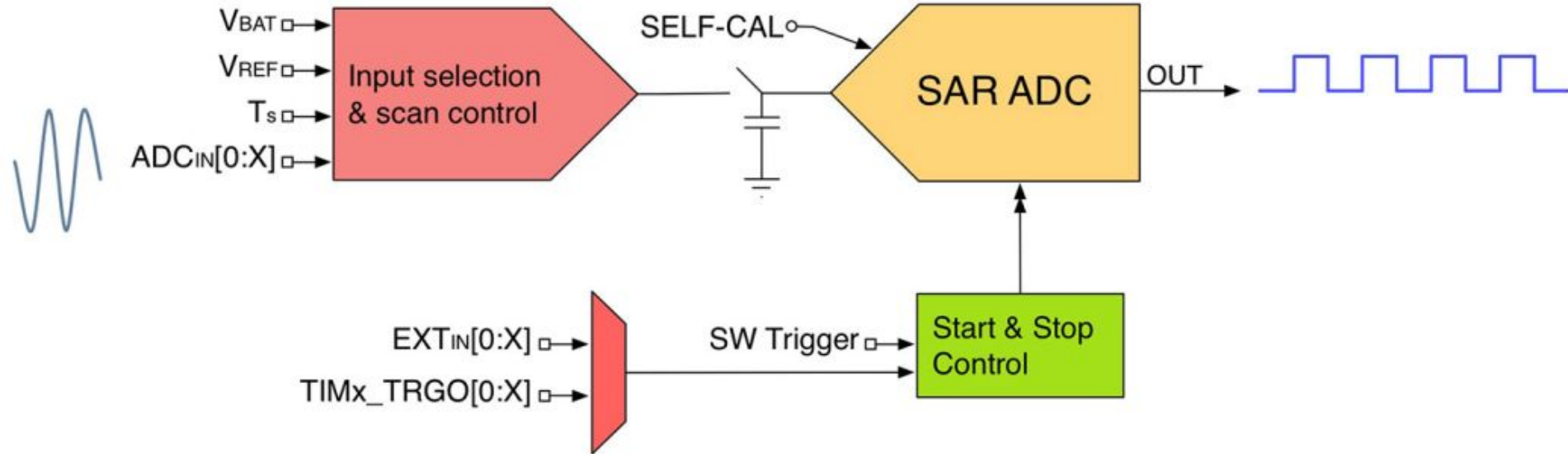
¿Qué es un ADC y cómo se usa en ST?

- Convierte señales analógicas en digitales
- La familia STM32F4 implementa:
12-bit Successive Approximation Register ADC
- $2^{12} = 4096$ niveles posibles
- HAL_ADC_MODULE
 stm32f4xx_hal_adc.c
 stm32f4xx_hal_adc.h

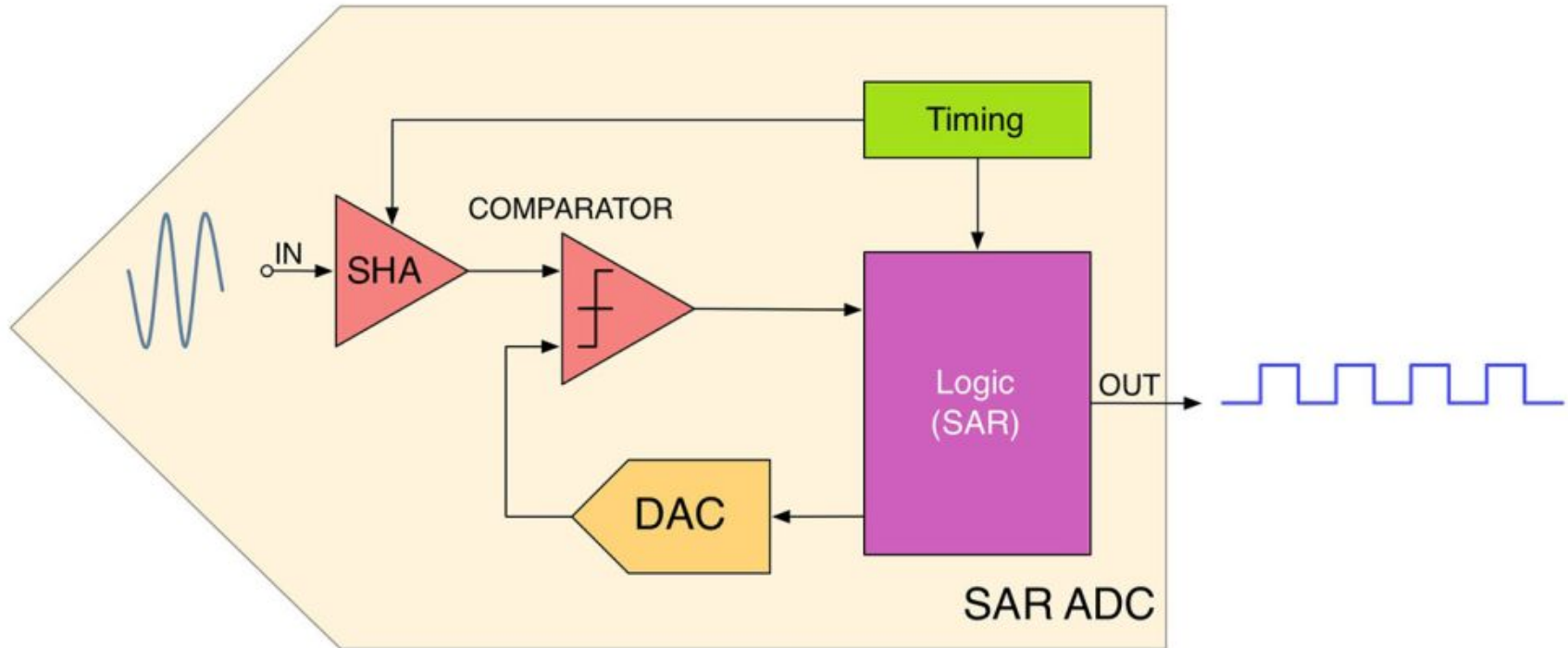
¿Cómo funciona un SAR ADC?

1. **Etapas de Muestreo y Retención:** la señal analógica de entrada se mantiene constante durante el proceso de conversión.
2. **Registro de Aproximación Sucesiva:** un registro de aproximación sucesiva (SAR) ajusta iterativamente su valor hasta que se aproxima lo suficiente a la señal analógica de entrada.
3. **Conversión Digital:** cada iteración del SAR compara la señal analógica con el valor digital actual y ajusta los bits del registro hasta que se alcanza la mejor aproximación posible.

Estructura simplificada de un ADC



Estructura interna simplificada de un SAR ADC



HAL_ADC Module

C struct ADC_HandleTypeDef,

```
typedef struct {  
    ADC_TypeDef      *Instance;           /* Pointer to ADC descriptor */  
    ADC_InitTypeDef  Init;               /* ADC initialization parameters */  
    __IO uint32_t     NbrOfCurrentConversionRank; /* ADC number of current conversion rank */  
    DMA_HandleTypeDef *DMA_Handle;       /* Pointer to the DMA Handler */  
    HAL_LockTypeDef   Lock;              /* ADC locking object */  
    __IO uint32_t     State;             /* ADC communication state */  
    __IO uint32_t     ErrorCode;         /* Error code */  
} ADC_HandleTypeDef;
```

```

typedef struct {
    uint32_t ClockPrescaler;      /* Selects the ADC clock frequency */
    uint32_t Resolution;          /* Configures the ADC resolution mode */
    uint32_t ScanConvMode;        /* The scan sequence direction. */
    uint32_t ContinuousConvMode;  /* Specifies whether the conversion is performed in
                                   Continuous or Single mode */
    uint32_t DataAlign;           /* Specifies whether the ADC data alignment
                                   is left or right */
    uint32_t NbrOfConversion;     /* Specifies the number of input that will be converted
                                   within the regular group sequencer */
    uint32_t NbrOfDiscConversion; /* Specifies the number of discontinuous conversions in
                                   which the main sequence of regular group */
    uint32_t DiscontinuousConvMode; /* Specifies whether the conversion sequence of regular
                                   group is performed in Complete-sequence/Discontinuous
                                   sequence */
    uint32_t ExternalTrigConv;     /* Select the external event used to trigger the start
                                   of conversion */
    uint32_t ExternalTrigConvEdge; /* Select the external trigger edge and enable it */
    uint32_t DMAContinuousRequests; /* Specifies whether the DMA requests are performed in
                                   one shot or in continuous mode */
    uint32_t EOCSelction;          /* Specifies what EOC (End Of Conversion) flag is used
                                   for conversion polling and interruption */
} ADC_InitTypeDef;

```

Modos de conversión ([AN3116](#))

- **Single-Channel, Single Conversion Mode**

La conversión se inicia mediante un evento de disparo (trigger) y el ADC se detiene después de completar la conversión.

Es útil para aplicaciones que requieren lecturas puntuales y no continuas

- **Multichannel (scan), single conversion mode**

El ADC escanea y convierte varias señales analógicas secuencialmente, pero realiza **una sola conversión por cada canal** en cada ciclo de escaneo. Después de completar la conversión de todos los canales configurados, el ADC se detiene.

Este modo es adecuado para aplicaciones que necesitan medir varias señales analógicas de forma secuencial pero no continua.

Modos de conversión ([AN3116](#))

- **Single-channel continuous conversion mode**

El ADC **convierte continuamente una única señal analógica**. Después de completar una conversión, el ADC inicia automáticamente la siguiente sin necesidad de un nuevo evento de disparo.

- **Multichannel (scan) continuous conversion mode**

El ADC escanea y convierte **varias señales analógicas de manera continua**. Después de completar una secuencia de conversiones para todos los canales configurados, el ADC reinicia automáticamente la secuencia

Modos de conversión ([AN 3116](#))

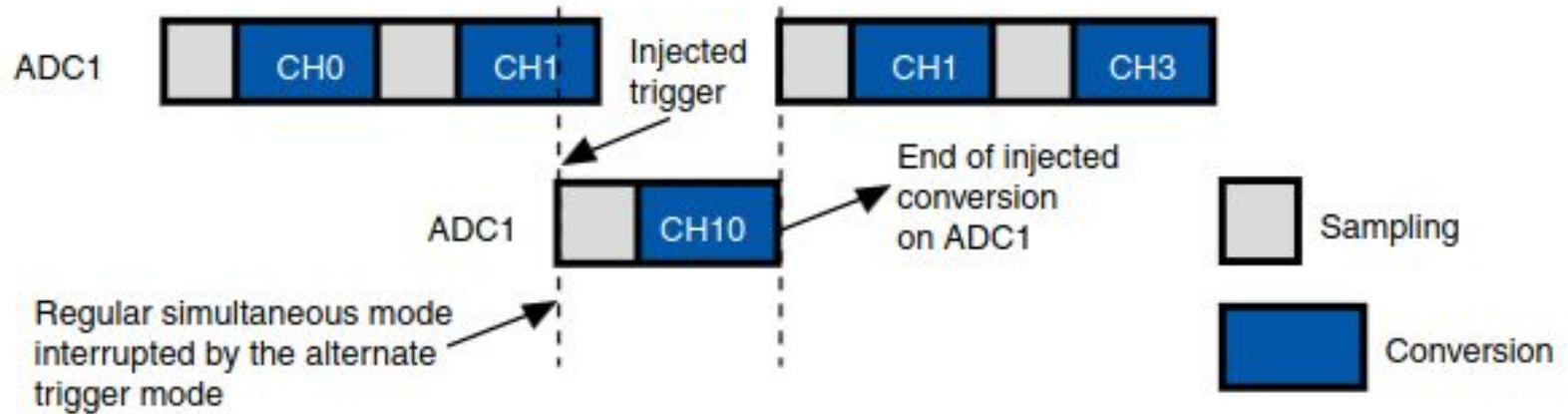
- **Injected conversion mode**

El ADC realiza conversiones de canales específicos en respuesta a un evento de disparo externo o por software, independientemente del modo de conversión regular.

Esto permite realizar **conversiones de alta prioridad** suspendiendo las conversiones regulares en curso.

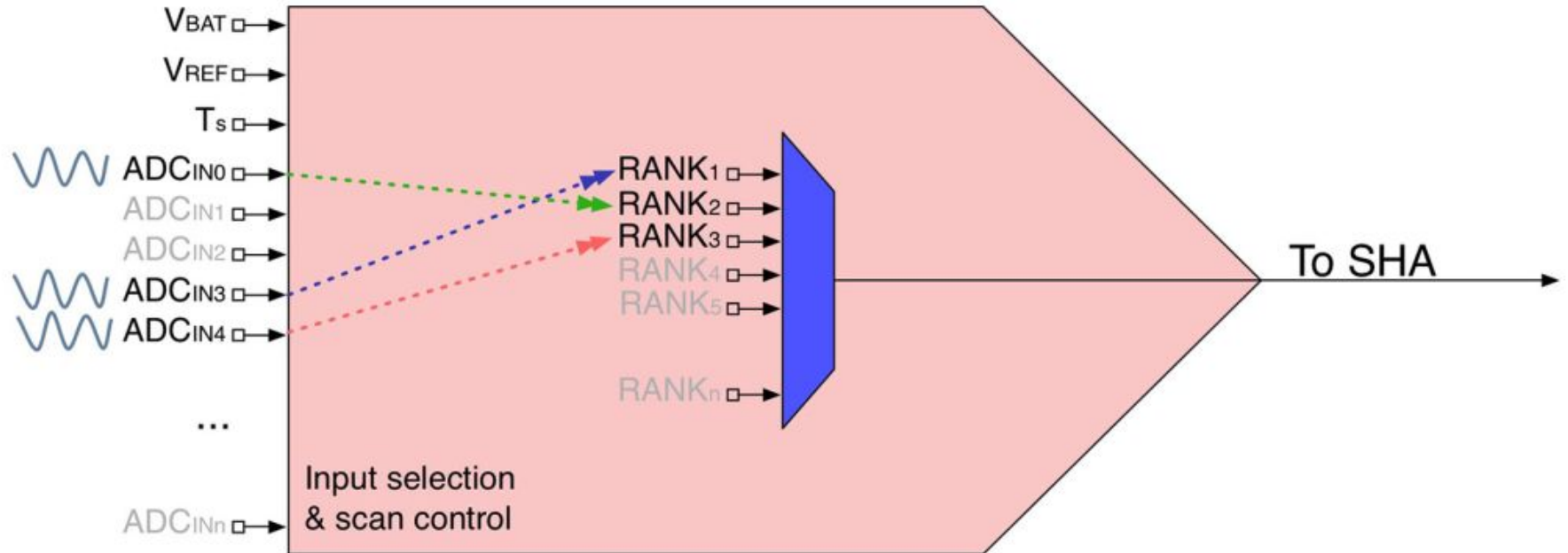
Es útil en aplicaciones donde ciertos eventos o condiciones requieren una medición rápida y precisa de canales específicos.

Injected conversion mode



Channel Selection

- Los input channels están fijos a pines de MCU.
- Se pueden ordenar lógicamente asignando un índice 1-16 (*rank*).



Channel/rank configuration

C struct ADC_Channel-ConfTypeDef

```
typedef struct {  
    uint32_t Channel;           /* Specifies the channel to configure into ADC rank */  
    uint32_t Rank;              /* Specifies the rank ID */  
    uint32_t SamplingTime;      /* Sampling time value for the selected channel */  
    uint32_t Offset;            /* Reserved for future use, can be set to 0 */  
} ADC_ChannelConfTypeDef;
```

Sobre el parámetro *SamplingTime*:

- Es el tiempo en el que el ADC muestrea la señal analógica antes de iniciar la conversión a digital.
- No puede tomar valores arbitrarios y se especifica en ciclos de reloj del ADC.

ADC Resolution and Conversion Speed

- Se puede convertir más rápido si se reduce la resolución.
- El tiempo de muestreo (*SamplingTime*) se compone de un número fijo (3 cycles) + un número variable que depende de la resolución:
- El tiempo mínimo de conversión para cada resolución es:
 - 12 bits: $3 + \sim 12 = 15$ ADCCLK cycles
 - 10 bits: $3 + \sim 10 = 13$ ADCCLK cycles
 - 8 bits: $3 + \sim 8 = 11$ ADCCLK cycles
 - 6 bits: $3 + \sim 6 = 9$ ADCCLK cycles

Modos de operación del ADC

1. Polling mode

```
HAL_StatusTypeDef HAL_ADC_Start(&adcHandler);  
HAL_StatusTypeDef HAL_ADC_PollForConversion(ADC_HandleTypeDef* hadc, uint32_t Timeout);  
uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef* adcHandler);
```

2. Interrupt mode

```
HAL_NVIC_SetPriority(ADC_IRQn, 0, 0);  
HAL_NVIC_EnableIRQ(ADC_IRQn);  
HAL_ADC_Start_IT(&adcHandler);  
  
void ADC_IRQHandler(void) {  
    HAL_ADC_IRQHandler(&adcHandler);  
}  
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* adcHandler) {  
    ...  
}
```

STM32F4xx_hal_adc.h

```
/* I/O operation functions *****/
HAL_StatusTypeDef HAL_ADC_Start(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_Stop(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_PollForConversion(ADC_HandleTypeDef* hadc, uint32_t Timeout);

HAL_StatusTypeDef HAL_ADC_PollForEvent(ADC_HandleTypeDef* hadc, uint32_t EventType, uint32_t Timeout);

HAL_StatusTypeDef HAL_ADC_Start_IT(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_Stop_IT(ADC_HandleTypeDef* hadc);

void HAL_ADC_IRQHandler(ADC_HandleTypeDef* hadc);

HAL_StatusTypeDef HAL_ADC_Start_DMA(ADC_HandleTypeDef* hadc, uint32_t* pData, uint32_t Length);
HAL_StatusTypeDef HAL_ADC_Stop_DMA(ADC_HandleTypeDef* hadc);

uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef* hadc);

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc);
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef* hadc);
void HAL_ADC_LevelOutOfWindowCallback(ADC_HandleTypeDef* hadc);
void HAL_ADC_ErrorCallback(ADC_HandleTypeDef *hadc);
```